

# Class XII CBSE Computer Science — E-Content

## GROUP BY & HAVING in SQL

CBSE Class XII Computer Science (083) | Interactive Learning Module

### 1. Learning Outcomes

- Explain why GROUP BY is used.
- Construct SQL queries using GROUP BY.
- Combine GROUP BY with aggregate functions.
- Explain the difference between WHERE and HAVING.
- Construct queries using GROUP BY ... HAVING.
- Predict the output of grouped queries.
- Identify common errors in GROUP BY queries.
- Solve CBSE-style SQL questions confidently.

### 2. Warm-up: Think Like a Data Analyst

Consider the following EMPLOYEE table:

| EmpID | Name  | Department | Salary |
|-------|-------|------------|--------|
| 101   | Anu   | HR         | 30000  |
| 102   | Ravi  | IT         | 50000  |
| 103   | Meera | HR         | 35000  |
| 104   | Arjun | IT         | 60000  |
| 105   | Diya  | Sales      | 40000  |
| 106   | Kiran | IT         | 55000  |
| 107   | Neha  | Sales      | 45000  |

Question: What is the average salary in each department? The question requires rows belonging to the same department to be combined. This is exactly what GROUP BY does.

### 3. The Big Idea

Without GROUP BY:

```
SELECT AVG(Salary)
FROM EMPLOYEE;
```

| AVG(Salary) |
|-------------|
| 45000       |

With GROUP BY:

```
SELECT Department, AVG(Salary)
FROM EMPLOYEE
GROUP BY Department;
```

| Department | AVG(Salary) |
|------------|-------------|
| HR         | 32500       |
| IT         | 55000       |
| Sales      | 42500       |

Remember: GROUP BY divides rows into groups based on common values. Think: Rows → Groups → Calculation on each group.

## 4. GROUP BY Simulation

Example student data:

| Student | House | Marks |
|---------|-------|-------|
| A       | Red   | 80    |
| B       | Blue  | 70    |
| C       | Red   | 90    |
| D       | Blue  | 60    |
| E       | Green | 85    |

```
SELECT House, AVG(Marks)
FROM STUDENT
GROUP BY House;
```

SQL conceptually identifies Red, Blue and Green groups, places rows into their respective groups, and calculates AVG(Marks) separately for each group.

| House | AVG(Marks) |
|-------|------------|
| Red   | 85         |
| Blue  | 65         |
| Green | 85         |

## 5. Syntax of GROUP BY

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

Example:

```
SELECT Department, COUNT(*)
FROM EMPLOYEE
GROUP BY Department;
```

| Department | COUNT(*) |
|------------|----------|
| HR         | 2        |
| IT         | 3        |
| Sales      | 2        |

## 6. GROUP BY + Aggregate Functions

| Function | Purpose            |
|----------|--------------------|
| COUNT()  | Counts values/rows |
| SUM()    | Calculates total   |
| AVG()    | Calculates average |
| MAX()    | Finds maximum      |
| MIN()    | Finds minimum      |

### Number of employees in each department

```
SELECT Department, COUNT(*)  
FROM EMPLOYEE  
GROUP BY Department;
```

### Total salary department-wise

```
SELECT Department, SUM(Salary)  
FROM EMPLOYEE  
GROUP BY Department;
```

### Average salary department-wise

```
SELECT Department, AVG(Salary)  
FROM EMPLOYEE  
GROUP BY Department;
```

### Highest salary in each department

```
SELECT Department, MAX(Salary)  
FROM EMPLOYEE  
GROUP BY Department;
```

### Lowest salary in each department

```
SELECT Department, MIN(Salary)  
FROM EMPLOYEE  
GROUP BY Department;
```

## 7. Quick Check #1

Which query calculates total sales for each city?

- A. SELECT City, SUM(Sales) FROM SALES;
- B. SELECT City, SUM(Sales) FROM SALES GROUP BY City;
- C. SELECT SUM(Sales) FROM SALES GROUP BY Sales;
- D. SELECT City FROM SALES GROUP BY SUM(Sales);

Answer: B — because we want one total for every City group.

## 8. Why Do We Need HAVING?

Suppose we want to display only those departments having more than 2 employees. The condition is applied to groups, not individual rows. Therefore, we use HAVING.

```
SELECT Department, COUNT(*)
FROM EMPLOYEE
GROUP BY Department
HAVING COUNT(*) > 2;
```

| Department | COUNT(*) |
|------------|----------|
| IT         | 3        |
| Finance    | 6        |

## 9. WHERE vs HAVING — Most Important Comparison

| WHERE                            | HAVING                                  |
|----------------------------------|-----------------------------------------|
| Filters individual rows          | Filters groups                          |
| Applied before grouping          | Applied after grouping                  |
| Example: Salary > 40000          | Example: AVG(Salary) > 40000            |
| Normally used for row conditions | Commonly used with aggregate conditions |

Memory trick: WHERE → Which ROWS? | HAVING → Which GROUPS?

## 10. Interactive Simulation: WHERE or HAVING?

| Question                                                 | Correct clause             | Reason                       |
|----------------------------------------------------------|----------------------------|------------------------------|
| Employees whose salary is greater than ₹50,000           | WHERE Salary > 50000       | Filters individual employees |
| Departments whose average salary is greater than ₹50,000 | HAVING AVG(Salary) > 50000 | Filters groups               |
| Departments having more than 3 employees                 | HAVING COUNT(*) > 3        | Filters groups               |
| Employees working in IT                                  | WHERE Department = 'IT'    | Filters individual records   |

## 11. GROUP BY + HAVING: Complete Example

| Product  | Category   | Price |
|----------|------------|-------|
| Pen      | Stationery | 10    |
| Pencil   | Stationery | 5     |
| Notebook | Stationery | 50    |
| Apple    | Food       | 30    |
| Mango    | Food       | 40    |
| Shirt    | Clothing   | 500   |
| Jeans    | Clothing   | 900   |

Question: Display categories whose average price is greater than 100.

```
SELECT Category, AVG(Price)
FROM PRODUCT
GROUP BY Category
HAVING AVG(Price) > 100;
```

| Category | AVG(Price) |
|----------|------------|
|----------|------------|

|          |     |
|----------|-----|
| Clothing | 700 |
|----------|-----|

## 12. A Very Important CBSE Pattern

Use this template when a question asks for a grouping column, an aggregate value, and a condition on that aggregate:

```
SELECT GroupColumn, AGGREGATE(Column)
FROM TableName
GROUP BY GroupColumn
HAVING AGGREGATE(Column) condition;
```

Example — departments whose maximum salary is greater than 50,000:

```
SELECT Department, MAX(Salary)
FROM EMPLOYEE
GROUP BY Department
HAVING MAX(Salary) > 50000;
```

## 13. WHERE + GROUP BY + HAVING Together

Considering only employees earning more than ₹40,000, display departments whose average salary is greater than ₹50,000.

```
SELECT Department, AVG(Salary)
FROM EMPLOYEE
WHERE Salary > 40000
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

Conceptual execution: TABLE → WHERE (filter rows) → GROUP BY (create groups) → calculate AVG → HAVING (filter groups) → final result.

Exam tip: Do not use HAVING Salary > 40000 when the intention is to filter individual employees before grouping. Use WHERE Salary > 40000.

## 14. GROUP BY with ORDER BY

Question: Display the number of employees in each department in descending order of employee count.

```
SELECT Department, COUNT(*)
FROM EMPLOYEE
GROUP BY Department
ORDER BY COUNT(*) DESC;
```

GROUP BY creates department groups; COUNT() calculates the number of employees; ORDER BY sorts the final result.

## 15. CBSE Exam Connection

The official CBSE Class XII 2023–24 Sample Question Paper included a question asking which SQL clauses should be used to find the highest salary in each department and display the results in descending order of salary. The concept tested is GROUP BY + ORDER BY.

```
SELECT Department, MAX(Salary)
FROM EMPLOYEES
GROUP BY Department
ORDER BY MAX(Salary) DESC;
```

## 16. CBSE Board-Paper Connection

The official CBSE Class XII Computer Science 2025 board paper included a grouped-query question involving MAX(WAGE), MIN(WAGE), TYPE and GROUP BY. This tests the idea that aggregate functions are calculated separately for each distinct group.

```
SELECT MAX(WAGE), MIN(WAGE), TYPE
FROM WORKER
GROUP BY TYPE;
```

## 17. CBSE-Style Question Bank

### Q1 — 1 Mark

Which clause is used to group rows having the same values?

Answer: GROUP BY.

### Q2 — 1 Mark

Which clause is generally used to impose a condition on groups?

Answer: HAVING.

### Q3 — 1 Mark

In SELECT Department, COUNT(\*) FROM EMPLOYEE GROUP BY Department, what does each result row represent?

Answer: One department group and the number of employees belonging to that department.

### Q4 — 2 Marks

Write an SQL query to display the department-wise average salary.

```
SELECT Department, AVG(Salary) FROM EMPLOYEE GROUP BY Department;
```

### Q5 — 2 Marks

Write an SQL query to display departments having more than 5 employees.

```
SELECT Department, COUNT(*) FROM EMPLOYEE GROUP BY Department HAVING COUNT(*) > 5;
```

### Q6 — 2 Marks

Differentiate between WHERE and HAVING.

WHERE filters individual records before grouping; HAVING filters groups after GROUP BY.

## 18. Predict the Output Challenge

| TYPE      | WAGE |
|-----------|------|
| Skilled   | 1000 |
| Skilled   | 1500 |
| Unskilled | 700  |
| Unskilled | 900  |
| Skilled   | 1200 |

```
SELECT TYPE, MAX(WAGE), MIN(WAGE)
FROM WORKER
GROUP BY TYPE;
```

| TYPE      | MAX(WAGE) | MIN(WAGE) |
|-----------|-----------|-----------|
| Skilled   | 1500      | 1000      |
| Unskilled | 900       | 700       |

## 19. Common Mistakes

Mistake 1: Selecting a non-grouped column with an aggregate.

```
SELECT Department, AVG(Salary)
FROM EMPLOYEE;
```

Correct:

```
SELECT Department, AVG(Salary)
FROM EMPLOYEE
GROUP BY Department;
```

Mistake 2: Using an aggregate condition in WHERE.

```
SELECT Department, COUNT(*)
FROM EMPLOYEE
WHERE COUNT(*) > 3
GROUP BY Department;
```

Correct:

```
SELECT Department, COUNT(*)
FROM EMPLOYEE
GROUP BY Department
HAVING COUNT(*) > 3;
```

Mistake 3: Using GROUP BY when no grouping is required. To find the highest salary in the entire table, use SELECT MAX(Salary) FROM EMPLOYEE;

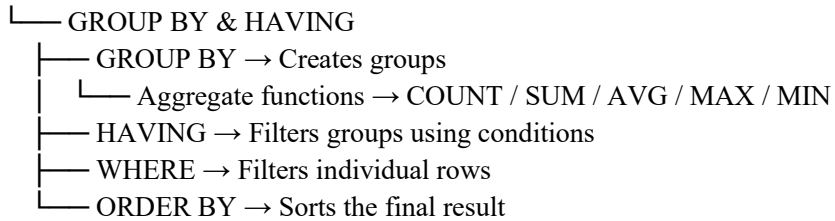
## 20. GROUP BY vs ORDER BY vs HAVING

| Clause   | Main question it answers     |
|----------|------------------------------|
| WHERE    | Which rows should I keep?    |
| GROUP BY | Which rows should I combine? |
| HAVING   | Which groups should I keep?  |

|          |                                            |
|----------|--------------------------------------------|
| ORDER BY | In what order should I display the result? |
|----------|--------------------------------------------|

## 21. Mind Map

SQL



## 22. Question → Clause Decoder

| Question wording                            | Likely SQL        |
|---------------------------------------------|-------------------|
| salary is...                                | WHERE             |
| department-wise / city-wise / category-wise | GROUP BY          |
| number of...                                | COUNT()           |
| total...                                    | SUM()             |
| average...                                  | AVG()             |
| highest...                                  | MAX()             |
| lowest...                                   | MIN()             |
| groups having...                            | HAVING            |
| ascending order                             | ORDER BY ... ASC  |
| descending order                            | ORDER BY ... DESC |

## 23. Rapid-Fire Assessment

**Which clause creates groups?** Answer: GROUP BY

**Which clause filters groups?** Answer: HAVING

**Which function counts records?** Answer: COUNT()

**Which function gives the average?** Answer: AVG()

**Which clause sorts the output?** Answer: ORDER BY

**Which comes first conceptually — WHERE or HAVING?** Answer: WHERE

**Can HAVING be used with GROUP BY?** Answer: Yes.

**Is GROUP BY required when finding the overall maximum?** Answer: No.

## 24. Board-Exam Master Practice Set

### Q1 — 1 Mark

Choose the correct query to display the number of students in each class.



- A. SELECT COUNT(\*) FROM STUDENT;
- B. SELECT Class, COUNT(\*) FROM STUDENT GROUP BY Class;
- C. SELECT Class FROM STUDENT HAVING COUNT(\*);
- D. SELECT Class, COUNT(\*) FROM STUDENT WHERE Class;

Answer: B

## Q2 — 2 Marks

Write a query to display the maximum salary department-wise.

```
SELECT Department, MAX(Salary)
FROM EMPLOYEE
GROUP BY Department;
```

## Q3 — 2 Marks

Write a query to display only those departments whose minimum salary is above ₹30,000.

```
SELECT Department, MIN(Salary)
FROM EMPLOYEE
GROUP BY Department
HAVING MIN(Salary) > 30000;
```

## Q4 — 3 Marks

Write a query to display the average marks subject-wise for subjects whose average is at least 60.

```
SELECT Subject, AVG(Marks)
FROM RESULT
GROUP BY Subject
HAVING AVG(Marks) >= 60;
```

## Q5 — 3 Marks

Write a query to display department-wise total salary in descending order.

```
SELECT Department, SUM(Salary)
FROM EMPLOYEE
GROUP BY Department
ORDER BY SUM(Salary) DESC;
```

## 25. Final Challenge

| EID | NAME | DEPT  | SALARY |
|-----|------|-------|--------|
| 1   | A    | IT    | 50000  |
| 2   | B    | HR    | 30000  |
| 3   | C    | IT    | 60000  |
| 4   | D    | HR    | 40000  |
| 5   | E    | IT    | 70000  |
| 6   | F    | SALES | 45000  |
| 7   | G    | SALES | 55000  |

a) Department-wise employee count.

```
SELECT DEPT, COUNT(*) FROM EMP GROUP BY DEPT;
```

b) Department-wise maximum salary.

```
SELECT DEPT, MAX(SALARY) FROM EMP GROUP BY DEPT;
```

c) Departments having more than 2 employees.

```
SELECT DEPT, COUNT(*) FROM EMP GROUP BY DEPT HAVING COUNT(*) > 2;
```

d) Departments having average salary greater than 50,000.

```
SELECT DEPT, AVG(SALARY) FROM EMP GROUP BY DEPT HAVING AVG(SALARY) > 50000;
```

e) Department-wise average salary in descending order.

```
SELECT DEPT, AVG(SALARY) FROM EMP GROUP BY DEPT ORDER BY AVG(SALARY) DESC;
```

## 26. One-Minute Revision

WHERE

↓

FILTER ROWS

GROUP BY

↓

MAKE GROUPS

HAVING

↓

FILTER GROUPS

ORDER BY

↓

SORT RESULT

**Golden Rule:** WHERE filters ROWS. GROUP BY creates GROUPS. HAVING filters GROUPS. ORDER BY sorts the RESULT.

## 27. CBSE Exam Strategy

1. Underline the grouping word: department-wise, city-wise, category-wise, type-wise → think GROUP BY.
2. Identify the calculation: total → SUM(), average → AVG(), maximum → MAX(), minimum → MIN(), number → COUNT().
3. Look for a condition on the calculated value: departments having average salary > 50000 → HAVING AVG(Salary) > 50000.
4. Look for sorting: descending order → ORDER BY ... DESC.
5. Construct the query in the usual sequence: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY.

## 28. Exit Ticket

A. Display city-wise total sales.

```
SELECT City, SUM(Sales)
FROM SALES
GROUP BY City;
```

B. Display cities whose total sales exceed ₹1,00,000.

```
SELECT City, SUM(Sales)
FROM SALES
GROUP BY City
HAVING SUM(Sales) > 100000;
```

C. Display the results in descending order of total sales.

```
ORDER BY SUM(Sales) DESC;
```

## 29. Suggested Interactive E-Learning Sequence

- Concept animation → table simulation → WHERE or HAVING interaction.
- GROUP BY query builder: arrange SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY.
- Output prediction: students predict results before revealing them.
- Error hunt: identify incorrect use of WHERE, GROUP BY and HAVING.
- Timed CBSE-style query construction.
- Mind-map revision and exit ticket.

**Core takeaway:** If the question asks you to calculate something group-wise, think GROUP BY. If it asks you to keep or reject groups based on an aggregate result, think HAVING.

**CBSE reference note:** The CBSE Academic website publishes official Class XII Sample Question Papers and Marking Schemes. The CBSE examination website publishes official board question papers. Verify the latest official papers when using this material for assessment.